

Semana 1 — Arquitetura do Godot, Nodes e Scene Tree

Semana 1

Arquitetura do Godot, Nodes e Scene Tree

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade I — Aprender a Ferramenta (Semanas 1–3)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender o papel de uma game engine na produção de um jogo
- Reconhecer Node e Scene Tree como unidade universal de composição
- Criar a estrutura inicial do projeto do Vertical Slice
- Construir a primeira Scene funcional com Nodes filhos

Semana 1 — Arquitetura do Godot, Nodes e Scene Tree

ENCONTRO 1

O Editor e a Estrutura do Projeto

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- O que é uma game engine e por que ela existe (20 min)
- Demonstração: tour pelo Godot Editor (35 min)
- Laboratório: estrutura do projeto do Vertical Slice, incluindo importação dos pacotes Kenney (50 min)
- Desafio: pasta adicional própria (20 min)
- Feedback e fechamento (10 min)

Semana 1 — Arquitetura do Godot, Nodes e Scene Tree



O que a Unity resolveu "por trás" dos GameObjects nos projetos que vocês já fizeram?

Pense antes de qualquer tela do Godot aparecer.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O que é uma Game Engine

Um jogo resolve, por baixo da jogabilidade, um conjunto fixo de problemas técnicos:

- **Renderização** — desenhar imagens a cada frame
- **Física** — simular colisão e movimento
- **Input** — traduzir teclado/controle em ação
- **Organização de cena** — estruturar o mundo do jogo
- **Assets** — carregar e gerenciar arte, som e dados

Por que uma Engine Existe

Uma game engine resolve essa camada **de forma reutilizável**, para que a equipe não precise reescrever tudo a cada novo jogo.

- É exatamente o papel que a Unity já cumpriu para vocês
- A diferença entre engines está em **como** cada uma organiza a solução — não em **se** ela existe
- Esse contraste (universal × implementação) se repete a cada novo conceito do semestre

Godot × Unity — Primeiro Contato

Godot

- Project Manager (gerenciador de projetos)
- Viewport (edição espacial)
- FileSystem Dock (assets do projeto)
- Scene = árvore de Nodes em `.tscn`

Unity

- Unity Hub
- Scene View
- Project window
- Scene = contêiner de GameObjects

Demonstração — Tour pelo Godot Editor

O que será mostrado:

- Abertura do Project Manager e criação de um novo projeto
- Abas **2D**, **3D**, **Script** e **AssetLib**
- O **Viewport** — área de edição espacial da cena
- O **FileSystem Dock** — todos os arquivos do projeto (`res://`)

Resultado esperado: a turma localiza Viewport e FileSystem Dock sem confundi-los.

Imagem sugerida

Objetivo: mostrar o Godot Editor aberto, com Viewport 3D e FileSystem Dock visíveis simultaneamente.

Enquadramento: captura de tela cheia do editor, com as duas áreas destacadas por retângulos coloridos.

*Elementos presentes: aba 3D ativa, Viewport central vazio, FileSystem Dock no canto inferior esquerdo listando **res://**.*

Destaque visual: contorno colorido separando Viewport (azul) de FileSystem Dock (verde).

Legenda sugerida: "Viewport e FileSystem Dock — as duas áreas centrais do primeiro contato com o Godot."

Laboratório — Estrutura do Vertical Slice

Cada estudante cria o projeto `TemploEsquecido` e organiza a estrutura de pastas:

```
res://  
├─ scenes/ (characters, interactables, ui, levels/exploration, levels/dungeon)  
├─ scripts/ (autoload, components)  
├─ orchestrations/  
├─ resources/ (items, save)  
├─ assets/ (dungeon, forest, characters)  
├─ materials/  
├─ audio/  
└─ animations/
```

Laboratório — Importação dos Pacotes Kenney

- Download de **Mini Dungeon**, **Mini Forest** e **Mini Characters** (kenney.nl, CC0)
- Cada pacote arrastado para sua subpasta em `assets/` (`dungeon/` , `forest/` , `characters/`)
- Os três pacotes ficam disponíveis desde hoje — cada um só é efetivamente usado quando o Cronograma exigir (texturas do Mini Dungeon na Semana 3, modelos do Mini Dungeon na Semana 5, Mini Characters na Semana 8, Mini Forest na Semana 12)

Boas Práticas — Organização de Projeto

- Nenhum asset solto na raiz de `res://`
- Toda pasta criada agora, mesmo vazia, evita retrabalho nas próximas 16 semanas
- Nunca mover ou renomear arquivos pelo sistema operacional — sempre pelo FileSystem Dock
- Nomenclatura consistente (snake_case para pastas e arquivos)



Desafio — Encontro 1

Organize uma pasta adicional dentro da estrutura do projeto (ex.: `prototypes` ou `references`) e justifique a escolha ao colega ao lado.

OBJETIVOS

Não há solução única — o objetivo é praticar organização própria dentro de uma convenção compartilhada.

Fechamento — Encontro 1

- Projeto Godot 4.7 criado e organizado
- Estrutura de pastas completa, pronta para receber conteúdo
- Próximo passo: a primeira Scene, com Nodes filhos, no Encontro 2

Semana 1 — Arquitetura do Godot, Nodes e Scene Tree

ENCONTRO 2

Node, Scene Tree e Composição

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (10 min)
- Composição × herança, Node e Scene Tree (20 min)
- Demonstração: Scene com Nodes filhos via Orchestrator (35 min)
- Laboratório: réplica da Scene (45 min)
- Desafio: Node filho adicional (20 min)
- Feedback e fechamento (5 min)

Conceito — Composição × Herança

Toda engine moderna resolve o mesmo problema: como montar um objeto de jogo a partir de partes menores.

- **Herança** — uma classe herda comportamento de outra
- **Composição** — um objeto reúne partes independentes, cada uma responsável por um pedaço do comportamento

O Godot resolve esse problema através de **Node** e **Scene Tree**.

Node e Scene Tree

- **Scene** — uma árvore de Nodes, salva em um arquivo `.tscn`
- **Node** — unidade básica de composição; cada um já nasce especializado (Node3D, MeshInstance3D, Light3D...)
- Um Node filho adicionado a outro é, na prática, um componente de comportamento
- A Scene Tree **é** a própria hierarquia de composição

Composição no Godot × Unity

Godot

- Node já nasce especializado
- Composição = organizar Nodes filhos
- A árvore inteira é a Scene

Unity

- GameObject nasce vazio
- Composição = anexar Components
- Components vivem "dentro" do GameObject

Diagrama Sugerido — Hierarquia da Scene

Diagrama sugerido

*Fluxo em árvore: **NivelTeste (Node3D)** no topo, com dois ramos filhos: **Chao (MeshInstance3D)** e **LuzPrincipal (DirectionalLight3D)**. Setas descendo do nó raiz para cada filho, sem setas entre os filhos (são independentes entre si).*

Objetivo: visualizar que a Scene Tree é uma árvore, não uma lista plana.

Legenda sugerida: "A Scene Tree é composição: cada filho é responsável por sua própria função."

Demonstração — Scene via Orchestrator

O que será construído:

- Scene `level_exploration.tscn` em `scenes/levels/exploration/`
- Nó raiz `NivelTeste` (Node3D)
- Filhos: `Chao` (MeshInstance3D) e `LuzPrincipal` (DirectionalLight3D)
- Orchestration `nivel_teste.torch` com um evento **Ready** conectado a um log

Por quê: primeira peça de conteúdo visível do semestre, base direta do Player na Semana 2.

Imagem sugerida

*Objetivo: mostrar a árvore de cena (Scene Tree dock) com **NivelTeste**, **Chao** e **LuzPrincipal** já organizados, ao lado do Viewport 3D mostrando o chão iluminado.*

Enquadramento: captura de tela dividida — Scene Tree dock à esquerda, Viewport 3D à direita.

Elementos presentes: hierarquia de três Nodes nomeados, malha visível e iluminada no Viewport.

Destaque visual: realce na relação pai-filhos na árvore.

Legenda sugerida: "Uma Scene simples: um Node3D pai com dois Nodes filhos especializados."

Laboratório — Réplica da Scene

Cada estudante replica, no próprio projeto:

1. Scene `level_exploration.tscn` com nó raiz `NivelTeste`
2. Filhos `Chao` e `LuzPrincipal`, renomeados e configurados
3. Orchestration `nivel_teste.torch` associada, com evento Ready testado
4. Teste da Scene isolada (F6), sem erros

Boas Práticas — Nomenclatura e Organização

- Nunca deixar Nodes com nome padrão do editor (`Node3D` , `MeshInstance3D2`)
- Sempre renomear para um nome que descreva a função
- Manter a Orchestration organizada desde o início, mesmo em grafos simples
- Seguir a convenção de nomenclatura do PROJECT_ARCHITECTURE.md



Desafio — Encontro 2

Adicione um Node filho adicional à Scene, não demonstrado em aula, produzindo um comportamento visual diferente do exemplo do professor.

OBJETIVOS

Escolha livre, desde que compatível com o escopo do Vertical Slice (ver PROJECT_ARCHITECTURE.md).

Resultado Esperado da Semana

- Projeto Godot 4.7 organizado, estrutura de pastas completa
- Três pacotes Kenney (Mini Dungeon, Mini Forest, Mini Characters) importados em `assets/`
- Scene funcional com ao menos três Nodes em hierarquia
- Um Node adicional criado de forma autônoma (desafio)
- Turma relaciona Node/Scene Tree ao par GameObject/Component da Unity

Checklist da Semana

- ☐ Projeto `TemploEsquecido` criado, abre sem erros
- ☐ Estrutura de pastas completa (scenes, scripts, orchestrations, resources, assets, materials, audio, animations)
- ☐ Kenney Mini Dungeon, Mini Forest e Mini Characters importados em `assets/`, sem erro de importação
- ☐ Scene `level_exploration.tscn` com `NivelTeste`, `Chao`, `LuzPrincipal`
- ☐ Orchestration `nivel_teste.torch` funcional
- ☐ Node adicional do desafio, distinto do exemplo

Próximos Passos — Semana 2

A Scene e a estrutura desta semana são a base direta da Semana 2:

- Adição de um **CharacterBody3D** como Node filho
- Locomoção via **move_and_slide**
- Configuração do **Input Map**

Leitura recomendada: Godot Docs — Nodes and Scenes; Unity Manual (consulta comparativa).